

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas Informáticos



Generación estructurada de archivos YAML a
partir de lenguaje natural mediante modelos
de lenguaje

TRABAJO DE FIN DE MÁSTER

Presentado para la obtención del título de Máster por:

Mario García Berenguer

Madrid, 2026



UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



Máster en Aprendizaje Automático y Datos Masivos

Generación estructurada de archivos YAML a partir de lenguaje natural mediante modelos de lenguaje

TRABAJO DE FIN DE MÁSTER

Presentado para la obtención del título de Máster por:

Mario García Berenguer

Bajo la supervisión de:
Dr. Javier Huertas Tato

Madrid, 2026

<Dedicatoria (opcional)>

Agradecimientos

<Página de agradecimientos (opcional)>

< Si el trabajo ha recibido financiación de alguna convocatoria competitiva, por favor, indíquelo aquí escribiendo: "Este trabajo ha sido parcialmente apoyada por...". De lo contrario, elimine este texto para dejar esta sección en blanco. >

Abstract

<Resumen en inglés: máximo de 4000 caracteres, texto plano (sin símbolos), resumen estructurado de la tesis (introducción o motivación, objetivos, hallazgos y conclusiones)>

Resumen

<Resumen en español: máximo de 4000 caracteres, texto plano (sin símbolos), resumen estructurado de la tesis (introducción o motivación, objetivos, hallazgos y conclusiones)>

Índice general

Agradecimientos	iii
Abstract	v
Resumen	vi
Índice de figuras	viii
Índice de tablas	ix
Abreviaturas y acrónimos	xii
1 Introduction	1
1.1 Motivación	1
1.2 Descripción	2
1.3 Objetivos	4
2 Estado del arte	5
2.1 Modelos de lenguaje autoregresivos y generación secuencial	5
2.2 De texto libre a artefactos técnicos estructurados	6
2.3 Fiabilidad de salidas estructuradas en LLMs	7
2.4 Control estructural: gramáticas, parsers y constrained decoding	8
2.5 Predicción estructurada y representación explícita de jerarquía	9
2.6 Adaptación supervisada eficiente: SFT, LoRA y QLoRA	10
2.7 Optimización por preferencias y alineamiento post-SFT	10
2.8 Kubernetes como dominio de evaluación estructurada	11
2.9 Síntesis y hueco identificado	12
3 Materiales y métodos	13
3.1 Título de la sección	13
3.1.1 Título de la subsección	13
4 Resultados	15
4.1 Título de la sección	15
4.1.1 Título de la subsección	15
5 Discusión	17
5.1 Título de la sección	17
5.1.1 Título de la subsección	17

6 Conclusiones	19
Anexos	27

Índice de figuras

Índice de tablas

Abreviaturas y acrónimos

UPM Universidad Politécnica de Madrid

Capítulo 1

Introduction

Esta plantilla sigue el formato de tesis de la *Universidad Politécnica de Madrid* (UPM), que se encuentra disponible[aquí](#).

En el siguiente capítulo se proporciona información acerca de las reglas para preparar el documento de la tesis, así como la explicación del uso de esta plantilla.

1.1 Motivación

La motivación de este trabajo surge frente a una necesidad natural de todo ingeniero, la automatización, especialmente de tareas repetitivas. Con el auge de los modelos de lenguaje de gran tamaño (LLMs), se ha abierto un campo totalmente inexplorado, en el que se posibilita la generación automática de texto a partir de instrucciones en lenguaje natural (prompting). Sin embargo, aunque esta capacidad es impresionante, se vuelve mucho más compleja y menos fiable cuando la salida esperada no es texto libre, sino un resultado que requiere de cierta estructura formal, no solo sintáctica, sino también jerárquica y semántica. Este problema se manifiesta claramente, por ejemplo, en la generación de manifiestos YAML de Kubernetes, donde una salida que puede parecer razonable desde el punto de vista lingüístico, puede ser inválida, inconsistente o directamente inutilizable desde el punto de vista técnico. En este contexto, no basta con que el modelo “entienda” el prompt: debe además construir una representación del resultado que respete la estructura requerida por el dominio.

En este sentido, la motivación técnica central de este trabajo se enmarca en el estudio de la generación estructurada de YAML a partir de lenguaje natural. El interés del problema surge, en parte, de las limitaciones que presentan las soluciones generalistas actuales. Un LLM convencional, utilizado únicamente mediante prompting o incluso tras un ajuste supervisado básico, genera la salida de manera autoregresiva token a token. Es decir, su mecanismo natural consiste en producir una secuencia plana de tokens, uno detrás de otro. Este mecanismo no ofrece, por sí mismo, garantías sobre la estructura global de la salida, como la que esperarías conseguir en un manifiesto YAML, que debe respetar una jerarquía de claves y valores, así como dependencias internas entre distintos componentes. A primera vista, parecería que los LLMs no serían las herramientas idóneas para este tipo de tareas, pero precisamente por

eso resulta interesante estudiar cómo pueden adaptarse o complementarse para mejorar su capacidad de generar estructuras formales correctas.

A partir de esta tensión, el problema puede plantearse desde varias aproximaciones, cada una con ventajas e inconvenientes. Por un lado, podríamos tratar este desafío como un problema de estilización del texto. Es decir, el modelo aprendería a generar una salida que, aunque es texto plano, sigue un formato específico que permita más tarde reconstruir de alguna forma determinista la estructura deseada. Esta estrategia es relativamente sencilla de implementar, pero tiene la limitación de que el modelo no tiene una señal explícita sobre la jerarquía, lo que puede dificultar la generación de estructuras complejas o profundas. Por otro lado, podríamos tratar de incorporar una señal estructural explícita durante el proceso de generación, por ejemplo, mediante un cabezal adicional que prediga cierta información sobre la jerarquía o la estructura del documento. Esta estrategia es más compleja de implementar, pero podría facilitar que el modelo aprenda a organizar la información de manera más coherente y estructurada.

La elección de este tema también está motivada por el interés de trabajar sobre un caso de estudio técnicamente relevante y al mismo tiempo evaluable de forma formal. Kubernetes resulta especialmente apropiado porque permite combinar lenguaje natural, estructuras jerárquicas y reglas verificables en un dominio real, ampliamente utilizado y con una semántica técnica suficientemente rica. En este sentido, el proyecto no solo persigue mejorar la capacidad de un modelo para producir archivos YAML, sino también construir una metodología reproducible para estudiar cómo se comportan los LLMs cuando la tarea exige algo más que fluidez textual.

Por último, existe una motivación personal que también resulta importante en la elección del tema. Más allá del interés por los modelos de lenguaje y por las tareas de NLP, este trabajo nace de la intención de abordar un problema asociado habitualmente a sistemas de tipo “caja negra” desde una perspectiva más analítica y, en cierta medida, más matemática. En lugar de limitarse a observar el comportamiento externo del modelo, el objetivo es estudiar el problema mediante métricas objetivas, restricciones formales, análisis estructural del dataset y experimentos que permitan aislar con mayor claridad dónde aparecen los errores y por qué aparecen. Esta motivación conecta con la idea de trasladar herramientas de análisis más rigurosas a un ámbito en el que con frecuencia predominan aproximaciones muy empíricas o poco interpretables. De este modo, el proyecto no solo pretende aportar una solución práctica a una tarea de generación estructurada, sino también servir como marco para estudiar con mayor profundidad el comportamiento de los LLMs en un problema concreto, técnico y medible.

1.2 Descripción

Este Trabajo Fin de Máster se centra en el estudio de la generación estructurada de configuraciones técnicas a partir de instrucciones en lenguaje natural mediante modelos de lenguaje de gran tamaño. En particular, el proyecto aborda como caso de estudio la generación automática de manifiestos de Kubernetes, un dominio especialmente adecuado por combinar una estructura jerárquica bien definida con restricciones sintácticas y semánticas que pueden validarse, al menos parcialmente, de forma automática.

A diferencia de las tareas de generación de texto libre, en este problema no basta con que el modelo produzca una salida lingüísticamente plausible. La salida debe respetar simultáneamente varios niveles de corrección: debe ser sintácticamente válida como YAML, mantener una estructura jerárquica coherente, utilizar claves y valores compatibles con el dominio y reflejar de forma fiel los requisitos expresados en el prompt. Además, en el caso concreto de Kubernetes, la calidad de la generación depende también de relaciones semánticas entre distintos componentes, como la correspondencia entre los selectores de un `Service` y las etiquetas de los pods que debe exponer, la presencia de campos básicos como `apiVersion`, `kind` o `metadata.name`, la definición correcta de contenedores e imágenes, o la coherencia entre volúmenes declarados y puntos de montaje usados por los contenedores.

El objetivo del proyecto es diseñar y evaluar un sistema capaz de transformar descripciones en lenguaje natural en manifiestos YAML estructurados, válidos y semánticamente consistentes dentro del dominio de Kubernetes. Para ello, el trabajo parte de la idea de que el problema debe abordarse no como una mera tarea de generación autoregresiva token a token, sino como una tarea en la que el modelo debe organizar el contenido del prompt dentro de una estructura formal. Viéndolo de esta forma, podemos entender este tipo de ficheros YAML como estructuras de árbol, donde cada entrada indentada ocupa una posición concreta dentro del documento final. Esta separación resulta especialmente relevante porque muchos de los errores observables en este tipo de tareas no provienen únicamente de una mala interpretación del prompt, sino también de la dificultad del modelo para serializar correctamente la información en un formato fuertemente restringido.

Metodológicamente, el proyecto se apoyará en un modelo base autoregresivo de tipo *decoder-only*, adaptado mediante técnicas de ajuste fino eficiente, como LoRA, sobre un conjunto de pares *prompt-salida*. A partir de esa base, se compararán dos estrategias principales para estudiar la robustez estructural del sistema. La primera, denominada en el repositorio `serialized_sft`, mantendrá la jerarquía en la superficie textual de salida: el modelo generará una representación serializada en la que cada línea contiene tanto el contenido YAML como su nivel jerárquico, y un parser determinista se encargará de reconstruir el manifiesto final sin inventar claves ni reparar errores semánticos de forma oculta.

La segunda estrategia, denominada `two_head_sft`, parte de una intuición distinta. Si el YAML se interpreta como un árbol proyectado sobre una secuencia de líneas, el orden de los nodos queda dado por el propio orden de generación del modelo. Lo que no queda determinado de forma fiable es la profundidad de cada entrada. Por ello, esta rama propone separar la predicción del contenido de la predicción de la jerarquía: el modelo generará el texto de cada línea, mientras que un cabezal estructural explícito predecirá el valor `level`. La comparación entre ambas ramas permitirá estudiar si la jerarquía debe aprenderse como una parte más de la secuencia generada o si conviene modelarla como una señal estructural diferenciada.

La evaluación del sistema se diseñará específicamente para el problema de generación estructurada. Por ello, no se tomarán como referencia principal métricas textuales superficiales, ya que dos configuraciones pueden diferir léxicamente y, sin embargo, ser funcionalmente equivalentes, o parecer similares en texto y ser estructuralmente incorrectas. En su lugar, se definirán métricas en varios niveles: métricas sintácticas, como el porcentaje de salidas YAML parseables; métricas estructurales, como la reconstrucción segura desde bloques, la

coincidencia de niveles `level` o la consistencia de la jerarquía; métricas de dominio Kubernetes, como la presencia de campos requeridos y la coherencia entre selectores, etiquetas, contenedores, puertos o volúmenes; y métricas de adecuación al prompt, destinadas a medir si la configuración generada refleja correctamente la intención del usuario. Esta evaluación se complementará con un análisis sistemático de errores para identificar patrones recurrentes y comprender mejor las limitaciones de cada estrategia.

1.3 Objetivos

El objetivo general de este Trabajo Fin de Máster es diseñar y evaluar un sistema basado en modelos de lenguaje capaz de generar manifiestos YAML de Kubernetes a partir de instrucciones en lenguaje natural, garantizando no solo la validez sintáctica de la salida, sino también su coherencia estructural y semántica respecto al prompt de entrada.

A partir de este objetivo general, se plantean los siguientes objetivos específicos:

- Analizar y caracterizar el dataset disponible, estudiando su tamaño, diversidad, complejidad estructural y posibles limitaciones para el entrenamiento del modelo.
- Diseñar un conjunto de métricas objetivas que permitan evaluar de forma rigurosa la calidad de las salidas generadas, considerando parseabilidad YAML, consistencia estructural, adecuación al prompt y validez de dominio Kubernetes.
- Definir una representación intermedia basada en bloques de línea, donde cada unidad contenga el texto de la línea y su nivel jerárquico `level`, de forma que la estructura pueda reconstruirse y evaluarse de manera explícita.
- Adaptar un modelo base de lenguaje al dominio de generación de configuraciones de Kubernetes mediante técnicas de ajuste fino eficientes.
- Comparar una estrategia de ajuste supervisado que serializa el nivel jerárquico como texto, `serialized_sft`, con una estrategia que incorpora un cabezal estructural explícito para predecir dicho nivel, `two_head_sft`.
- Evaluar experimentalmente el sistema propuesto sobre un conjunto de prueba representativo, identificando fortalezas, limitaciones y patrones de error tanto en el contenido generado como en la jerarquía predicha.
- Extraer conclusiones sobre la utilidad de los mecanismos de control estructural en tareas de generación técnica, y sobre la conveniencia de tratar la jerarquía como una parte más de la superficie textual o como una señal estructural diferenciada.

Capítulo 2

Estado del arte

La generación automática de manifiestos de Kubernetes a partir de instrucciones en lenguaje natural se sitúa en una zona intermedia entre varios campos de investigación. A primera vista, podría entenderse como una tarea más de generación de texto: dado un prompt, el modelo debe producir una respuesta. Sin embargo, esta lectura se queda corta, porque la salida esperada no es una explicación en lenguaje natural, sino un artefacto técnico con una estructura formal. Un manifiesto YAML puede escribirse como texto plano, pero su validez depende de una jerarquía de claves, listas, campos, recursos y relaciones internas.

Por este motivo, este capítulo no revisa la literatura como una sucesión aislada de técnicas, sino como una cadena de problemas cada vez más concretos. Primero se introduce la generación autoregresiva en modelos de lenguaje y su capacidad para seguir instrucciones. Después se analiza el salto desde el texto libre hacia artefactos estructurados, especialmente código, lenguajes específicos de dominio y salidas con formato. A continuación se revisan mecanismos de control estructural, predicción estructurada y adaptación supervisada eficiente. Finalmente, se sitúa el caso de Kubernetes dentro de la literatura sobre configuraciones técnicas y se identifica el hueco que motiva la comparación central de este trabajo: aprender la jerarquía como parte de la superficie textual de salida o predecirla como una variable estructural explícita.

2.1 Modelos de lenguaje autoregresivos y generación secuencial

Los modelos de lenguaje modernos se apoyan en gran medida en la arquitectura Transformer, introducida por Vaswani et al. como una alternativa basada exclusivamente en mecanismos de atención para modelar dependencias en secuencias [42]. Esta arquitectura permitió escalar el entrenamiento sobre grandes corpus y favoreció el desarrollo de modelos capaces de capturar regularidades lingüísticas, patrones de razonamiento y conocimiento de dominio a partir de datos masivos. Desde entonces, los modelos de lenguaje de gran tamaño han consolidado un paradigma en el que la misma arquitectura general puede adaptarse a un conjunto amplio de tareas mediante prompting, ajuste supervisado o alineamiento posterior [51].

La propiedad central para este trabajo es que estos modelos generan de forma autoregresiva. Es decir, producen una secuencia token a token, condicionando cada nueva decisión al prompt inicial y a los tokens ya generados. Esta formulación es natural para texto libre, donde la salida puede interpretarse como una continuación lingüística. Sin embargo, cuando el resultado esperado es una estructura formal, la generación secuencial introduce una tensión: el modelo debe decidir localmente el siguiente token, pero la corrección de la salida depende de propiedades globales, como la apertura y cierre de estructuras, la consistencia entre campos o la profundidad jerárquica de cada línea.

El desarrollo de modelos ajustados para seguir instrucciones ha reducido parte de esta distancia entre el objetivo de preentrenamiento y el comportamiento esperado en uso real. Trabajos como FLAN muestran que el ajuste con tareas formuladas como instrucciones mejora la generalización a tareas no vistas [44], mientras que Self-Instruct explora la generación automática de instrucciones como forma de ampliar datos de entrenamiento [43]. A su vez, las revisiones sobre prompting muestran que la forma de presentar la tarea al modelo influye de manera decisiva en el resultado [30]. Estas líneas explican por qué un modelo instruct puede ser un punto de partida razonable para transformar una petición en lenguaje natural en una salida técnica.

No obstante, seguir instrucciones no equivale a garantizar estructura. El modelo puede entender que el usuario solicita un **Deployment**, un **Service** o un **CronJob**, y aun así fallar al serializar correctamente la jerarquía de campos que define ese recurso. Precisamente por eso, el modelo base utilizado en este proyecto, perteneciente a la familia Qwen2.5 [34], debe entenderse como una política autoregresiva potente, pero no como una solución completa al problema de generación jerárquica. El reto no es únicamente producir una respuesta plausible, sino hacer que esa respuesta respete una estructura que el mecanismo autoregresivo no representa de forma explícita.

2.2 De texto libre a artefactos técnicos estructurados

La dificultad aparece con mayor claridad cuando se compara la generación de texto libre con la generación de código o de configuraciones técnicas. En una respuesta abierta, puede haber muchas formulaciones aceptables. En cambio, en un programa, una consulta SQL, un fichero de configuración o un manifiesto de Kubernetes, pequeños errores de sintaxis o de estructura pueden hacer que la salida sea inutilizable. Por ello, la literatura sobre generación de código ha insistido en evaluar los modelos no solo mediante similitud textual, sino mediante criterios funcionales o ejecutables [8]. Esta idea resulta especialmente relevante para el presente trabajo: dos manifiestos pueden ser textualmente distintos y equivalentes en intención, o parecer muy similares y diferir en un detalle estructural que invalida el resultado.

Los modelos especializados en código, como Code Llama, muestran que el entrenamiento sobre corpus técnicos mejora la capacidad de generar artefactos formales y razonar sobre ellos [38]. Sin embargo, YAML y los lenguajes de configuración ocupan una posición algo distinta a la de los lenguajes de programación generalistas. No siempre tienen una semántica operacional inmediata como Python o Java, pero sí imponen esquemas, convenciones y relaciones entre campos. Son, en cierto sentido, lenguajes de descripción técnica: expresan una configuración

que otro sistema interpretará posteriormente.

Este matiz aparece en trabajos orientados a lenguajes específicos de dominio. DocCGen, por ejemplo, estudia la generación controlada de código en dominios como Ansible YAML y comandos Bash, destacando que los DSLs estructurados presentan dificultades específicas por su gramática, su esquema y su dependencia de documentación de dominio [32]. De forma similar, Ansible Wisdom aborda la generación de tareas YAML de Ansible desde lenguaje natural y propone métricas específicas para este tipo de automatización [33]. Estos trabajos son cercanos al planteamiento del TFM porque también conectan lenguaje natural con artefactos técnicos serializados, aunque el dominio Kubernetes añade una capa adicional de dificultad por la variedad de recursos, campos anidados y relaciones entre objetos.

La consecuencia para este proyecto es que la generación de manifiestos no debe plantearse como una simple tarea de redacción. El modelo no solo debe escoger palabras, sino organizar una configuración. Esta organización se manifiesta finalmente como YAML, pero su corrección depende de una estructura subyacente. Por eso, antes de discutir cómo adaptar el modelo, conviene revisar qué se sabe sobre la fiabilidad de los LLMs cuando se les exige producir salidas estructuradas.

2.3 Fiabilidad de salidas estructuradas en LLMs

En los últimos años se ha extendido el uso de LLMs para generar objetos con formato: JSON, XML, YAML, CSV, llamadas a herramientas o respuestas ajustadas a un esquema. Esta tendencia responde a una necesidad práctica: integrar modelos de lenguaje en sistemas que esperan salidas procesables automáticamente. Sin embargo, la literatura reciente muestra que la fiabilidad de estas salidas sigue siendo un problema abierto. StructEval, por ejemplo, evalúa la capacidad de los modelos para generar y convertir múltiples formatos estructurados, incluyendo YAML, y muestra que la adherencia al formato y la corrección estructural no pueden darse por supuestas incluso en modelos avanzados [48].

Algo similar ocurre en benchmarks centrados en JSON y esquemas. JSONSchemaBench analiza la generación restringida por esquemas reales y muestra que la complejidad del esquema afecta tanto a la cobertura de los métodos de constrained decoding como a la calidad final de las respuestas [15]. StructuredRAG, por su parte, estudia la capacidad de los modelos para producir respuestas JSON fiables en tareas de recuperación y generación, subrayando que seguir instrucciones de formato no elimina por completo los fallos de consistencia [26]. StructBench llega a una conclusión parecida desde la generación de datos estructurados complejos: los errores no se limitan a detalles superficiales, sino que afectan a campos, restricciones y relaciones internas [28].

Estos resultados son importantes porque separan dos niveles que a menudo se confunden. Una salida puede ser parseable y, sin embargo, no respetar el esquema esperado. También puede respetar parcialmente el esquema y aun así no representar correctamente la intención del prompt. En el caso de YAML, esta distinción es todavía más visible: la indentación y las listas dan forma textual a una estructura de árbol, pero la validez sintáctica del YAML no garantiza que el manifiesto sea correcto en Kubernetes. Por tanto, la evaluación del presente trabajo

debe considerar varios niveles: parseabilidad YAML, reconstrucción estructural desde bloques, precisión del nivel jerárquico, validez aproximada de dominio y adecuación al prompt.

La literatura sobre salidas estructuradas permite formular una primera conclusión parcial: no basta con pedir al modelo que devuelva un formato. El formato puede mejorar la procesabilidad, pero no garantiza que la estructura interna se haya aprendido de forma robusta. A partir de aquí aparecen dos grandes familias de soluciones: restringir la generación mediante mecanismos externos, o modificar la forma en que el modelo aprende y representa la estructura.

2.4 Control estructural: gramáticas, parsers y constrained decoding

Una respuesta natural a los fallos de formato consiste en restringir el espacio de salida. Si el modelo puede escoger cualquier token en cada paso, nada impide que produzca una secuencia inválida. Los métodos de constrained decoding introducen restricciones léxicas, gramaticales o semánticas para bloquear continuaciones que no pertenecen al lenguaje objetivo. Grid Beam Search, por ejemplo, extiende el beam search para imponer restricciones léxicas en generación secuencial [18]. En dominios más formales, el uso de gramáticas permite controlar no solo palabras concretas, sino la forma completa de la salida [6].

En tareas de semantic parsing también se ha explorado cómo adaptar modelos de lenguaje a espacios de salida estructurados. Shin et al. muestran que los modelos de lenguaje pueden utilizarse como semantic parsers cuando se les guía hacia un sublenguaje controlado que después se transforma en una representación formal [40]. Más recientemente, la gramática se ha incorporado directamente al proceso de decoding. Grammar-Constrained Decoding plantea que muchas tareas estructuradas pueden formularse como generación bajo una gramática y muestra mejoras frente a generación no restringida [16].

PICARD es especialmente relevante para este trabajo porque utiliza parsing incremental para restringir decodificadores autoregresivos [39]. La intuición es cercana a la de este TFM: el modelo sigue generando secuencialmente, pero un mecanismo externo verifica si las continuaciones parciales son compatibles con una estructura formal. Otros trabajos, como Efficient Guided Generation y Guidance, exploran enfoques más generales para controlar la generación mediante restricciones, programas o esquemas de salida [45, 4].

En este proyecto, el parser cumple una función relacionada, aunque no idéntica. No se utiliza como un reparador semántico ni como una herramienta que invente contenido ausente, sino como una frontera de control estructural. Su entrada no es YAML libre, sino una secuencia de bloques con texto de línea y nivel jerárquico. A partir de esa representación, reconstruye el manifiesto final aplicando indentación de forma determinista. Esta decisión reduce parte de la fragilidad del formato, pero también deja claro un límite: si el modelo predice mal el contenido o el nivel, el parser no debe ocultar ese fallo mediante reparaciones implícitas.

2.5 Predicción estructurada y representación explícita de jerarquía

La segunda respuesta al problema consiste en tratar la estructura no solo como una restricción externa, sino como una parte explícita de la tarea de aprendizaje. Esta idea tiene una tradición larga en predicción estructurada. A diferencia de una clasificación independiente, donde cada etiqueta puede evaluarse por separado, la predicción estructurada considera salidas compuestas por partes dependientes entre sí [21]. Los Structured Prediction Energy Networks continúan esta línea al modelar dependencias entre componentes de la salida mediante una función de energía aprendida [3]. También los modelos de transición con normalización global muestran que, en tareas como parsing o etiquetado, la decisión local debe entenderse dentro de una trayectoria estructural completa [2].

Esta perspectiva encaja de forma natural con YAML. Un manifiesto puede verse como un árbol serializado en líneas. La generación autoregresiva proporciona el orden: línea 0, línea 1, línea 2, y así sucesivamente. Pero ese orden no determina por sí mismo la altura de cada línea dentro de la jerarquía. En YAML, la profundidad se expresa mediante indentación, y pequeños cambios en esa profundidad pueden alterar por completo la interpretación del documento. Por ello, el valor `level` utilizado en este proyecto no es una anotación cosmética, sino una variable estructural que hace observable una parte de la jerarquía.

La literatura sobre representaciones lingüísticas también resulta útil para matizar esta idea. Los structural probes muestran que parte de la información sintáctica puede recuperarse de las representaciones internas de modelos neuronales [17], y los análisis de atención en BERT sugieren que ciertos patrones de atención se alinean con dependencias sintácticas o relaciones lingüísticas [10]. Sin embargo, que una estructura sea recuperable o esté implícita no significa que sea conveniente dejarla siempre sin supervisión explícita. En una tarea técnica, donde un error de jerarquía invalida el resultado, puede ser razonable convertir esa estructura en una señal de entrenamiento diferenciada.

Algo parecido se observa en generación de código y semantic parsing. Abstract Syntax Networks generan salidas siguiendo la estructura de árboles de sintaxis abstracta [35], mientras que los modelos sintácticos de generación de código incorporan reglas gramaticales para producir programas bien formados [49]. Estos trabajos no son equivalentes a la generación de YAML Kubernetes, pero sí refuerzan una intuición común: cuando la salida tiene una estructura formal, modelar explícitamente esa estructura puede ser más robusto que confiar únicamente en la superficie textual.

Desde esta posición se entiende mejor la hipótesis del cabezal de nivel. La rama serializada pregunta hasta dónde puede llegar un modelo que aprende la jerarquía como texto. La rama con cabezal, en cambio, plantea que la jerarquía merece una señal supervisada propia. Si el modelo debe aprender simultáneamente el contenido de cada línea y su posición en el árbol YAML, la separación entre predicción textual y predicción estructural puede ofrecer una forma más clara de estudiar los errores. En caso de emplear pérdidas múltiples para contenido y estructura, la literatura sobre ponderación de pérdidas en aprendizaje multi-tarea ofrece además un marco para razonar sobre el equilibrio entre objetivos [24].

2.6 Adaptación supervisada eficiente: SFT, LoRA y QLoRA

Una vez definido el problema como generación estructurada, el siguiente paso es adaptar el modelo al dominio. El ajuste supervisado, o SFT, es la etapa más directa cuando se dispone de ejemplos positivos de entrada y salida. En lugar de optimizar una recompensa externa, el modelo aprende a imitar demostraciones del formato deseado. Esta estrategia es especialmente adecuada al estado actual del proyecto, porque el dataset procesado contiene pares de prompt y YAML normalizado, así como una representación estructural derivada en bloques con `level`.

Ahora bien, ajustar completamente un LLM grande suele ser costoso. Por ello, las técnicas de fine-tuning eficiente resultan relevantes. LoRA propone congelar los pesos principales del modelo e introducir matrices de bajo rango entrenables, reduciendo de forma significativa el número de parámetros actualizados [20]. QLoRA extiende esta idea al entrenamiento sobre modelos cuantizados en 4 bits, permitiendo adaptar modelos grandes con menor consumo de memoria [11]. Estas técnicas hacen viable experimentar con modelos instruct en entornos académicos o de recursos limitados, sin abandonar por completo la capacidad del modelo base.

El papel del SFT en este trabajo no es únicamente mejorar el estilo de la salida. Es, más bien, una forma de estudiar dos formulaciones supervisadas del mismo problema. En `serialized_sft`, la salida objetivo contiene tanto el texto de cada línea como el valor `level` serializado. El modelo debe aprender la jerarquía como parte de la secuencia textual. En `two_head_sft`, el contenido de la línea y el nivel jerárquico se separan, de modo que el modelo conserva una tarea de generación textual, pero añade una tarea estructural explícita.

Esta comparación se apoya también en la literatura reciente sobre post-entrenamiento. El ajuste supervisado sigue apareciendo como una fase necesaria antes de aplicar métodos de preferencias más complejos [44, 19, 47]. Incluso cuando se estudian limitaciones de generalización del SFT, la conclusión práctica no suele ser saltarse esta fase, sino entender mejor qué puede y qué no puede aprender una política a partir de demostraciones [46]. Para este TFM, esto implica que la comparación supervisada entre `serialized_sft` y `two_head_sft` debe ser el núcleo experimental antes de introducir cualquier alineamiento posterior.

2.7 Optimización por preferencias y alineamiento post-SFT

El alineamiento mediante preferencias surge cuando la imitación de una única respuesta de referencia resulta insuficiente. En RLHF, el modelo no se optimiza directamente contra una salida canónica, sino contra un modelo de recompensa entrenado a partir de comparaciones humanas [31, 25]. Desde un punto de vista conceptual, esto permite ordenar respuestas alternativas según preferencias, pero introduce una nueva fragilidad: la recompensa utilizada durante la optimización es un proxy aprendido, no una medida perfecta de la calidad real.

Los análisis críticos de RLHF destacan precisamente esa tensión. El reward model puede

sufrir sesgos, falta de cobertura fuera de distribución y vulnerabilidad a reward hacking [7, 23]. Además, optimizar demasiado una señal de preferencias puede producir un coste de alineamiento, degradando capacidades previas del modelo [29]. RLAIIF intenta reducir el coste de recopilar preferencias humanas sustituyéndolas parcial o totalmente por feedback generado por modelos, pero esto desplaza el problema hacia la fiabilidad y los sesgos del evaluador artificial [27].

Frente a la complejidad de PPO y de los flujos clásicos de RLHF, han aparecido métodos de optimización directa de preferencias. DPO reformula el problema para ajustar el modelo directamente sobre pares preferidos y rechazados, evitando entrenar explícitamente un reward model separado y aplicar RL online [36]. Trabajos posteriores comparan DPO con PPO, estudian su robustez ante feedback ruidoso y proponen variantes como ORPO, GPO o KTO [47, 9, 19, 41, 13]. También se han revisado formulaciones de tipo REINFORCE para entender qué parte de la complejidad de RLHF es realmente necesaria [1, 12].

En el contexto de este TFM, esta literatura debe leerse con cautela. El proyecto no presupone una tubería completa de RLHF humano, ni un modelo de recompensa entrenado con anotadores, ni un sistema online de optimización de políticas. La conexión relevante es más concreta: una vez obtenida una política supervisada razonablemente competente, se pueden construir preferencias automáticas a partir de señales de validez sintáctica, reconstrucción estructural, adecuación al prompt o comprobaciones aproximadas de dominio. Por ello, DPO aparece como una posible fase post-SFT, no como sustituto de la comparación supervisada principal.

2.8 Kubernetes como dominio de evaluación estructurada

El caso de Kubernetes permite aterrizar las ideas anteriores en un dominio real. Kubernetes utiliza manifiestos YAML para describir recursos como `Pod`, `Deployment`, `Service`, `ConfigMap` o `CronJob`. Aunque el manifiesto se escriba como texto, el sistema que lo consume espera objetos con campos específicos, relaciones internas y convenciones de dominio. Esta diferencia es esencial: un YAML puede ser sintácticamente válido y, aun así, no describir un recurso de Kubernetes correcto o seguro.

La literatura sobre configuraciones Kubernetes confirma que este dominio es propenso a errores. Los estudios empíricos sobre defectos de configuración muestran que los manifiestos contienen fallos variados y que las herramientas estáticas no cubren todas las categorías de defecto [50]. Otros trabajos se centran en *misconfigurations* de seguridad en manifiestos open source, destacando que los errores de configuración pueden tener consecuencias prácticas graves [37]. También se han estudiado fallos de red y escenarios de fallo más amplios en Kubernetes, lo que refuerza la idea de que la corrección del dominio va más allá de parsear YAML [5, 14].

Este contexto justifica una evaluación por niveles. Primero, la salida debe ser parseable como YAML. Después, debe poder reconstruirse de forma segura desde la representación de bloques sin que el parser repare errores ocultos. Además, debe respetar una jerarquía coherente y emplear campos compatibles con Kubernetes. Finalmente, debe reflejar el prompt

de entrada. La existencia de herramientas de análisis de configuraciones, como Diffy, muestra que incluso fuera del aprendizaje profundo hay interés en detectar defectos en configuraciones estructuradas más allá de la sintaxis superficial [22].

En consecuencia, Kubernetes no funciona aquí solo como una aplicación ilustrativa. Es el dominio que hace visible la tensión central del trabajo. La generación autoregresiva proporciona una secuencia; YAML exige una jerarquía; Kubernetes añade restricciones de dominio. El pipeline propuesto en el repositorio, `prompt ->representación latente ->bloques con level ->parser ->YAML final`, responde precisamente a esa combinación de problemas.

2.9 Síntesis y hueco identificado

La literatura revisada ofrece tres respuestas complementarias al problema de generar salidas técnicas estructuradas. La primera consiste en adaptar el modelo mediante prompting, instruction tuning, SFT y fine-tuning eficiente. La segunda introduce restricciones externas mediante gramáticas, parsers o guided generation. La tercera optimiza preferencias después del ajuste supervisado, intentando favorecer salidas más útiles o válidas. Todas ellas son relevantes para este trabajo, pero ninguna resuelve por sí sola la pregunta específica que se plantea aquí.

El hueco aparece al observar cómo se representa la estructura. Muchas aproximaciones tratan la estructura como una propiedad del texto final: si el modelo produce una cadena que cumple el formato, la salida se considera estructurada. Sin embargo, en YAML la estructura no es solo una convención superficial. La indentación determina relaciones de pertenencia y profundidad. En Kubernetes, además, esa estructura contiene objetos de dominio con restricciones propias. Por tanto, aprender a escribir una cadena parecida a YAML no equivale necesariamente a aprender la jerarquía que hace que el YAML sea interpretable y útil.

Este TFM se sitúa precisamente en esa grieta. Por un lado, evalúa hasta dónde puede llegar un modelo autoregresivo cuando la jerarquía se serializa como texto, mediante la rama `serialized_sft`. Por otro, estudia si conviene separar la jerarquía como una señal supervisada explícita, mediante la rama `two_head_sft`. El parser actúa como control estructural común a ambas ramas, y las métricas se diseñan para observar no solo la similitud textual, sino la parseabilidad, la reconstrucción, la predicción de `level`, la adecuación al prompt y la validez aproximada de dominio.

La pregunta que queda abierta tras el estado del arte no es, por tanto, si un LLM puede producir texto con apariencia de YAML. La pregunta relevante es si puede aprender de forma fiable la organización jerárquica que convierte esa secuencia de líneas en un manifiesto estructuralmente válido. Esa pregunta conecta de manera directa con la metodología del trabajo, donde el dataset, la representación por bloques, el parser y las dos ramas de SFT se diseñan para hacer observable esta diferencia.

Capítulo 3

Materiales y métodos

3.1 Título de la sección

3.1.1 Título de la subsección

Capítulo 4

Resultados

4.1 Título de la sección

4.1.1 Título de la subsección

Capítulo 5

Discusión

5.1 Título de la sección

5.1.1 Título de la subsección

Capítulo 6

Conclusiones

Bibliografía

- [1] Ahmadian, A., Cremer, C., Galle, M., Fadaee, M., Kreutzer, J., Pietquin, O., Ustun, A., Hooker, S., 2024. Back to basics: Revisiting reinforce-style optimization for learning from human feedback in llms, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. URL: <https://aclanthology.org/2024.acl-long.662/>.
- [2] Andor, D., Alberti, C., Weiss, D., Severyn, A., Presta, A., Ganchev, K., Petrov, S., Collins, M., 2016. Globally normalized transition-based neural networks. arXiv preprint arXiv:1603.06042 URL: <https://arxiv.org/abs/1603.06042>.
- [3] Belanger, D., McCallum, A., 2017. Structured prediction energy networks, in: Proceedings of the 34th International Conference on Machine Learning. URL: <https://proceedings.mlr.press/v70/belanger17a.html>.
- [4] Beurer-Kellner, L., Fischer, M., Vechev, M., 2023. Guidance: A programming paradigm for controlling language models. arXiv preprint arXiv:2306.05285 URL: <https://arxiv.org/abs/2306.05285>.
- [5] Bufalino, J., Martin-Navarro, J.L., Di Francesco, M., Aura, T., 2025. Inside job: Defending kubernetes clusters against network misconfigurations. arXiv preprint arXiv:2506.21134 URL: <https://arxiv.org/abs/2506.21134>.
- [6] Bunel, R., Hausknecht, M., Devlin, J., Singh, R., Kohli, P., 2018. Leveraging grammar and reinforcement learning for neural program synthesis, in: International Conference on Learning Representations. URL: <https://openreview.net/forum?id=H1Xw62kRZ>.
- [7] Chaudhari, S., Aggarwal, P., Murahari, V.S., Rajpurohit, T., Kalyan, A., Narasimhan, K.R., Deshpande, A., Castro da Silva, B., 2025. Rlhf deciphered: A critical analysis of reinforcement learning from human feedback for llms. ACM Computing Surveys 58. URL: <https://doi.org/10.1145/3743127>, doi:10.1145/3743127.
- [8] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I.,

- Zaremba, W., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 URL: <https://arxiv.org/abs/2107.03374>.
- [9] Chowdhury, S.R., Kini, A., Natarajan, N., 2024. Provably robust dpo: Aligning language models with noisy feedback. arXiv preprint arXiv:2403.00409 URL: <https://arxiv.org/abs/2403.00409>.
 - [10] Clark, K., Khandelwal, U., Levy, O., Manning, C.D., 2019. What does bert look at? an analysis of bert’s attention, in: Proceedings of the 2019 ACL Workshop BlackboxNLP. URL: <https://arxiv.org/abs/1906.04341>.
 - [11] Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L., 2023. Qlora: Efficient finetuning of quantized llms, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/2305.14314>.
 - [12] Dong, H., Xiong, W., Pang, B., Wang, H., Zhao, H., Zhou, Y., Jiang, N., Sahoo, D., Xiong, C., Zhang, T., 2024. Rlhf workflow: From reward modeling to online rlhf. Transactions on Machine Learning Research URL: <https://openreview.net/forum?id=a13aYUU9eU>.
 - [13] Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., Kiela, D., 2024. Kto: Model alignment as prospect theoretic optimization. arXiv preprint arXiv:2402.01306 URL: <https://arxiv.org/abs/2402.01306>.
 - [14] Fonseca, J., et al., 2024. Mutiny! how does kubernetes fail, and what can we do about it? arXiv preprint arXiv:2404.11169 URL: <https://arxiv.org/abs/2404.11169>.
 - [15] Geng, S., Cooper, H., Moskal, M., et al., 2025. Generating structured outputs from language models: Benchmark and studies. arXiv preprint arXiv:2501.10868 URL: <https://arxiv.org/abs/2501.10868>.
 - [16] Geng, S., Josifoski, M., Peyrard, M., West, R., 2023. Grammar-constrained decoding for structured nlp tasks without finetuning, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2305.13971>.
 - [17] Hewitt, J., Manning, C.D., 2019. A structural probe for finding syntax in word representations, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1906.04284>.
 - [18] Hokamp, C., Liu, Q., 2017. Lexically constrained decoding for sequence generation using grid beam search, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://aclanthology.org/P17-1141/>.
 - [19] Hong, J., Lee, N., Thorne, J., 2024. Orpo: Monolithic preference optimization without reference model, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. URL: <https://aclanthology.org/2024.emnlp-main.626/>.
 - [20] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., 2021. Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 URL: <https://arxiv.org/abs/2106.09685>.

-
- [21] Joachims, T., Hofmann, T., Yue, Y., Yu, C.N., 2009. Predicting structured objects with support vector machines. *Communications of the ACM* .
- [22] Kakarla, S.K.R., Yan, F.Y., Beckett, R., 2024. Diffy: Data-driven bug finding for configurations. *Proceedings of the ACM on Programming Languages* 8. URL: <https://doi.org/10.1145/3656385>, doi:10.1145/3656385.
- [23] Kaufmann, T., Weng, P., Bengs, V., Huellermeier, E., 2023. A survey of reinforcement learning from human feedback. *arXiv preprint arXiv:2312.14925* URL: <https://arxiv.org/abs/2312.14925>.
- [24] Kendall, A., Gal, Y., Cipolla, R., 2018. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. URL: <https://arxiv.org/abs/1705.07115>.
- [25] Lambert, N., 2026. Reinforcement Learning from Human Feedback. URL: <https://rlhfbook.com>.
- [26] Lee, B., et al., 2024a. Structuredrag: Json response formatting with large language models. *arXiv preprint arXiv:2408.11061* URL: <https://arxiv.org/abs/2408.11061>.
- [27] Lee, H., Phatale, S., Mansoor, H., Mesnard, T., Ferret, J., Lu, K., Bishop, C., Hall, E., Carbune, V., Rastogi, A., Prakash, S., 2024b. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback, in: *Proceedings of the 41st International Conference on Machine Learning*.
- [28] Li, M., et al., 2024. Struc-bench: Are large language models really good at generating complex structured data?, in: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*. URL: <https://arxiv.org/abs/2309.08963>.
- [29] Lin, Y., Lin, H., Xiong, W., Diao, S., Liu, J., Zhang, J., Pan, R., Wang, H., Hu, W., Zhang, H., Dong, H., Pi, R., Zhao, H., Jiang, N., Ji, H., Yao, Y., Zhang, T., 2024. Mitigating the alignment tax of rlhf, in: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. URL: <https://aclanthology.org/2024.emnlp-main.35/>.
- [30] Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G., 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys* 55. URL: <https://doi.org/10.1145/3560815>, doi:10.1145/3560815.
- [31] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P.F., Leike, J., Lowe, R., 2022. Training language models to follow instructions with human feedback, in: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2203.02155>.
- [32] Pimparkhede, S., Kammakomati, M., Tamilselvam, S., Kumar, P., Pon Kumar, A., Bhattacharyya, P., 2024. Doccgen: Document-based controlled code generation, in:

- Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2406.11925>.
- [33] Pujar, S., Buratti, L., Guo, X., Dupuis, N., Lewis, B., Suneja, S., Sood, A., Nalawade, G., Jones, M., Morari, A., Puri, R., 2023. Automated code generation for information technology tasks in yaml through large language models. arXiv preprint arXiv:2305.02783 URL: <https://arxiv.org/abs/2305.02783>.
 - [34] Qwen Team, 2024. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 URL: <https://arxiv.org/abs/2412.15115>.
 - [35] Rabinovich, M., Stern, M., Klein, D., 2017. Abstract syntax networks for code generation and semantic parsing, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1704.07535>.
 - [36] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C., 2023. Direct preference optimization: Your language model is secretly a reward model, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/2305.18290>.
 - [37] Rahman, A., et al., 2023. Security misconfigurations in open source kubernetes manifests: An empirical study. ACM Transactions on Software Engineering and Methodology URL: <https://akondrahman.github.io/files/papers/tosem-k8s.pdf>.
 - [38] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Canton Ferrer, C., Grattafiori, A., Xiong, W., Defossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., Synnaeve, G., 2023. Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950 URL: <https://arxiv.org/abs/2308.12950>.
 - [39] Scholak, T., Schucher, N., Bahdanau, D., 2021. Picard: Parsing incrementally for constrained auto-regressive decoding from language models, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. URL: <https://arxiv.org/abs/2109.05093>.
 - [40] Shin, R., Lin, C.H., Thomson, S., Chen, C., Roy, S., Platanios, E.A., Pauls, A., Klein, D., Eisner, J., Van Durme, B., 2021. Constrained language models yield few-shot semantic parsers, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. URL: <https://aclanthology.org/2021.emnlp-main.608/>.
 - [41] Tang, Y., Guo, D.Z., Zheng, Z., Calandriello, D., Munos, R., Rowland, M., Richemond, P.H., Valko, M., Pires, B.A., Piot, B., 2024. Generalized preference optimization: A unified approach to offline alignment, in: Proceedings of the 41st International Conference on Machine Learning. URL: <https://proceedings.mlr.press/v235/tang24b.html>.
 - [42] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need, in: Advances in Neural Information Processing Systems. URL: <https://arxiv.org/abs/1706.03762>.
 - [43] Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N.A., Khashabi, D., Hajishirzi, H., 2023.

- Self-instruct: Aligning language models with self-generated instructions, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/2212.10560>.
- [44] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D., 2022. Scaling instruction-finetuned language models, in: International Conference on Learning Representations. URL: <https://arxiv.org/abs/2210.11416>.
- [45] Willard, B.T., Louf, R., 2023. Efficient guided generation for large language models. arXiv preprint arXiv:2307.09702 URL: <https://arxiv.org/abs/2307.09702>.
- [46] Wu, Y., Zhou, Y., Ziheng, Z., Peng, Y., Ye, X., Hu, X., Zhu, W., Qi, L., Yang, M.H., Yang, X., 2025. On the generalization of sft: A reinforcement learning perspective with reward rectification. OpenReview preprint URL: <https://openreview.net/forum?id=Lv7PjbcaMi>.
- [47] Xu, S., Fu, W., Gao, J., Ye, W., Liu, W., Mei, Z., Wang, G., Yu, C., Wu, Y., 2024. Is dpo superior to ppo for llm alignment? a comprehensive study, in: Proceedings of the 41st International Conference on Machine Learning. URL: <https://arxiv.org/abs/2404.10719>.
- [48] Yang, J., Jiang, D., He, L., Siu, S., Zhang, Y., Liao, D., Li, Z., Zeng, H., Jia, Y., Wang, H., Schneider, B., Ruan, C., Ma, W., Lyu, Z., Wang, Y., Lu, Y., Do, Q.D., Jiang, Z., Nie, P., Chen, W., 2025. Structeval: Benchmarking llms' capabilities to generate structural outputs. arXiv preprint arXiv:2505.20139 URL: <https://arxiv.org/abs/2505.20139>.
- [49] Yin, P., Neubig, G., 2017. A syntactic neural model for general-purpose code generation, in: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics. URL: <https://arxiv.org/abs/1704.01696>.
- [50] Zhang, Y., Paul, U., d'Amorim, M., Rahman, A., 2025. Configuration defects in kubernetes. arXiv preprint arXiv:2512.05062 URL: <https://arxiv.org/abs/2512.05062>.
- [51] Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.Y., Wen, J.R., 2023. A survey of large language models. arXiv preprint arXiv:2303.18223 URL: <https://arxiv.org/abs/2303.18223>.

Anexos

Incluye este apartado solo si es necesario. En caso contrario, comenta en el fichero principal (main.tex) las líneas `\appendix` y `\includecuerpo/x-Anexos` para evitar su inclusión.